Unité 12 — Projet de synthèse (énoncé)

Livrable B/C — Énoncé étudiant · UE « Software Security »
Code UE 7532 14 U32 D1 · Version révisée · 13 octobre 2026

Place dans le programme. Projet final évalué : identifier et expliquer des vulnérabilités courantes, choisir des contre-mesures et démontrer une démarche de développement sécurisé. Trois sujets au choix, de complexité croissante.

1. Principe et modalités

Chaque étudiant choisit un seul des trois niveaux ci-dessous. Les trois permettent d'obtenir la note maximale : la qualité de l'analyse, des preuves et des corrections prime sur la complexité technique.

• Travail : individuel, sauf indication contraire de l'enseignant.

• Environnement : application locale et isolée ; les manipulations offensives ne concernent que l'application autorisée.

• Preuves : commandes, résultats de tests et corrections reproductibles ; sources et outils cités.

2. Trois sujets au choix

NIVEAU 1 — Débutant
Audit et correction d'une application fournie

Point de départ : Une mini-application Flask vulnérable et son environnement Docker sont fournis.
Travail attendu : Identifier 2 vulnérabilités, en démontrer au moins 1 de manière contrôlée, proposer et appliquer les corrections, puis refaire les tests.
Outils possibles : Flask, curl, Bandit (ou outil équivalent). Rapport : 4 à 5 pages.

NIVEAU 2 — Intermédiaire
Sécurisation d'une mini-API existante

Point de départ : Le squelette d'une API de gestion de tâches (Flask + SQLite) est fourni.
Travail attendu : Identifier et corriger 3 vulnérabilités, décrire les risques principaux, écrire des tests de non-régression et justifier les protections.
Outils possibles : Flask, SQLite, Bandit, pytest (ou équivalents). Rapport : 6 à 8 pages.

NIVEAU 3 — Avancé
Conception et audit d'une petite application

Point de départ : Concevoir une petite API ou application (gestion de notes, tâches, carnet d'adresses...).
Travail attendu : Présenter les besoins et un modèle de menace simple ; implémenter l'application ; détecter, expliquer et corriger au moins 2 vulnérabilités ; employer 2 méthodes de test adaptées au langage.
Outils possibles : SAST, tests dynamiques, sanitizers ou fuzzing, selon pertinence. Rapport : 8 à 10 pages.

3. Livrables communs

• Code et exécution : sources, consignes de lancement et tests reproductibles (Docker si fourni ou utilisé).

• Rapport court : vulnérabilités et impacts, preuves avant/après, correctifs, limites et références.

• Démonstration orale : 5 à 10 minutes : présenter une faille, sa correction et le résultat du nouveau test.

• Classification : associer chaque faille à une CWE pertinente et estimer simplement sa gravité ; CVSS chiffré facultatif au niveau 1, attendu pour au moins une faille aux niveaux 2 et 3.

4. Évaluation — barème commun

Critère	Points

Compréhension et explication des vulnérabilités	25 %
Démarche d’analyse et preuves de tests	25 %
Pertinence des corrections et re-tests	30 %
Clarté du rapport et de la démonstration	20 %

Important : le niveau choisi n’apporte ni bonus ni pénalité. Les attentes sont ajustées au sujet ; la rigueur du travail détermine la note.

5. Jalons suggérés (cours du soir)

S1 : choix du niveau, prise en main et périmètre. S2 : analyse et premiers tests. S3 : corrections et tests complémentaires. S4 : vérification et rédaction. S5 : remise et démonstration.
Note : les applications de départ et consignes techniques des niveaux 1 et 2 seront fournies séparément par l’enseignant.